Code of Conduct:

I S.Kamalesh (192521013) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Kamalesh

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection. **Answer:** **Approach:** Random testing depends on tester intuition, so coverage is unknown and results cannot be repeated. Treating testing as an engineering activity means planning, designing, executing and measuring tests in a disciplined way. • Analyse requirements and identify test conditions and risks. • Design test cases using techniques such as equivalence partitioning, boundary value analysis and decision tables. • Define entry/exit criteria, expected results and traceability to requirements. • Execute, log defects, and measure coverage and defect trends. **How it improves quality:** Each requirement is covered by at least one designed test, so gaps are visible. Tests are repeatable and reviewable, which gives consistent results and finds more defects with fewer tests. Example: for a signup form, designed tests cover valid, invalid and boundary inputs instead of whatever the tester happens to type.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples. **Answer:** **Concept:** A defined testing process has planning, test design, execution, defect reporting and closure, with clear roles, entry/exit criteria and documentation. **Benefits in a banking system:** Standard steps make sure critical flows (fund transfer, balance update, interest calculation) are never skipped. • Early detection: a requirement review catches a wrong interest rule before coding. • Consistency: every release follows the same regression suite, so old failures do not return. • Traceability: each defect maps to a requirement and a test, which helps audits and compliance. • Reduced risk: transaction integrity, concurrency and security tests are planned, not accidental. **Example:** A transfer that debits one account but fails to credit the other is caught by a planned test case that checks both balances, preventing financial loss and loss of customer trust.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects.	3

Sl. No	Questions	Bloom's Level
	Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes. **Answer:** **Structured testing in SDLC:** Testing should be planned alongside each development phase (V-model idea): • Requirements: review requirements, prepare acceptance test plan. • Design: review design, prepare integration and system test cases. • Coding: unit testing and code reviews by developers. • Integration: test module interfaces such as cart to payment to inventory. • System/UAT: end-to-end, performance, security and user acceptance tests. • Maintenance: regression testing after every change. **Process discipline:** Defined entry/exit criteria, documented test cases and defect tracking make sure no phase is skipped. Integration and system defects, such as an order placed without stock update, are found at the right level and earlier, which lowers rework cost and improves release quality.	
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples. **Answer:** **Definitions:** Error (mistake): a human action that produces something incorrect. Defect (fault/bug): a flaw in the code or document caused by an error. Failure: the observable deviation of actual from expected behaviour when the defect is executed. **Classification:** The login mismatch between expected and actual output is a failure, because it is observed during execution. Its cause is a defect in the code, introduced by a developer's error. **Relationship:** Error leads to defect, and defect leads to failure (only when that code path runs). **Example:** Developer mistakenly writes if(password != stored) as if(password == stored) (error). This wrong line is the defect. A valid user being rejected at login is the failure.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example. **Answer:** **Principle:** Early testing: testing activities should start as early as possible in the SDLC. **How it helps:** Defects found in requirements or design are far cheaper to fix than those found in system testing or production. Reviews, static analysis and early test design expose ambiguities and logic errors before code is built on them. **Example:** If a review finds that 'discount' is undefined in the requirement, it is fixed with a document edit costing minutes. Found after release, it needs redesign, recoding, retesting and redeployment. Applying early testing therefore reduces cost, rework and delay and increases defect detection efficiency.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3

Sl. No	Questions	Bloom's Level
	Answer: **Tester's contribution in requirement phase:** The tester reviews the requirements and asks 'how will I test this?' • Identify ambiguous, incomplete, conflicting or untestable statements (for example, 'system should respond quickly'). • Raise clarification questions and propose measurable criteria (response time under 2 seconds). • Check traceability and identify risks such as patient data privacy and drug dosage rules. • Prepare early acceptance test conditions and test data. **Importance of early involvement:** In healthcare, a misunderstood requirement such as a dosage unit can cause serious harm. Early involvement removes ambiguity at its source, avoids defects in design and code, reduces cost, and improves communication between stakeholders.	
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues. **Answer:** **How defects originate:** Most defects start in the requirement phase and are then carried forward. • Requirements: ambiguous statements like 'calculate salary with allowances' without formula, tax slab or rounding rules. • Design: designers interpret the unclear requirement in their own way. • Coding: developers implement the assumed logic, so the defect is built in. • Testing: test cases written from the same unclear text give wrong expected results and miss the defect. **Prevention through clarity:** Requirements should be complete, unambiguous, consistent and testable, with worked examples (for example, basic pay 30,000 + HRA 20% - PF 12% = net pay). Requirement reviews, walkthroughs and traceability matrices make sure all roles share one understanding, so payroll outputs are correct.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue. **Answer:** **Classification:** This is a functional defect (input handling/boundary condition defect). The system does not behave as specified for valid or limit values, and the failure is a crash. It is typically a logic error such as using < instead of <=, or an unhandled limit. **Justification:** The failure occurs only at the edges of the input domain, which indicates inadequate boundary coverage in test design. **How BVA could have prevented it:** Boundary Value Analysis tests at and around each boundary: min-1, min, min+1, max-1, max, max+1. For an age field of 18 to 60, tests would use 17, 18, 19, 59, 60, 61. These values would have revealed the crash during testing, because most defects cluster at boundaries.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality. **Answer:**	3

Sl. No	Questions	Bloom's Level
	Concept: A defect repository stores each defect with ID, module, severity, root cause, phase found, steps to reproduce and status. **Support for test case design:** Analysis of the repository shows which modules, input types and scenarios produce defects most often. • Add new test cases for every defect type that escaped earlier. • Add past failures to the regression suite so they cannot recur silently. • Focus extra tests on defect-prone modules and use root-cause data to improve checklists. **Role in quality:** It converts past mistakes into lessons, reduces repeated defects across releases, supports metrics (defect density, leakage) and helps managers plan effort. Example: repeated date-format defects lead to a standard set of date test cases in every release.	
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented. **Answer:** **Identification:** Incorrect handling of transaction logic (for example the balance is not checked, or the account is debited but cash is not dispensed) means the system does not do what the specification requires. This is a functional defect. **Description:** Functional defects violate the specified behaviour of a feature. Here, the withdrawal function gives wrong results under certain conditions such as insufficient balance, daily limit exceeded or network failure mid-transaction. **Prevention:** • Review requirements for all withdrawal rules and exceptions. • Design test cases with decision tables, boundary values (balance equal to amount) and state-transition tests. • Add negative, rollback and concurrency scenarios and run regression tests. • Conduct code reviews of the transaction module.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency. **Answer:** **Defect clustering:** A small number of modules usually contain most of the defects (the Pareto 80/20 observation). **Identifying priorities:** • Analyse defect data by module and rank modules by defect density. • Identify complex, frequently changed or new modules as high risk. • Allocate more test effort, deeper techniques and earlier review to the top few modules. • Keep lighter coverage for stable modules, but monitor them. **Efficiency:** Effort is spent where defects are most likely, so more defects are found per hour of testing, time and cost are saved, and risk is reduced. Example: if payment and checkout hold 70% of defects, they receive most of the test cases.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution. **Answer:**	3

Sl. No	Questions	Bloom's Level
	Classification: Slow performance under heavy user load is a non-functional defect, specifically a performance (load/scalability) defect. Functionality is correct, but the quality attribute 'performance' is not met. **Techniques:** Defects can be classified by type (functional, performance, usability, security), severity (critical to low), priority, phase of origin and root cause (for example inefficient query, no caching). **How classification helps:** It routes the defect to the right owner (database or infrastructure team), sets the right priority, helps choose the test approach (load and stress testing), supports trend analysis and speeds up resolution.	
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency. **Answer:** **Reuse of historical defect data:** Past defects show where the software is fragile. • Select regression test cases that cover modules with frequent defects. • Convert each fixed defect into a regression test to confirm that it stays fixed. • Prioritise tests by defect severity and by areas affected by recent changes. • Remove redundant tests that never find defects. **Benefit:** Coverage is improved because risky areas and known failure patterns are always checked. Efficiency is improved because the suite is risk-based and smaller, which shortens regression cycles while still catching regressions.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps. **Answer:** **Role of a tester:** A tester works as a bridge between stakeholders and developers. • Participate in requirement, design and code reviews and clarify interface specifications. • Share test cases and test data with developers early. • Report defects clearly, with steps to reproduce, logs and severity, in a common tracker. • Join daily stand-ups and defect triage meetings. **Early involvement:** Interface mismatches and assumptions (for example data format between modules) are found at design stage, before integration. Shared understanding reduces misunderstandings, blame and rework, so fewer defects appear in integration testing.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality. **Answer:** **Defect prevention techniques:** Prevention aims to stop defects from being introduced. • Root cause analysis (for example 5 Whys, fishbone) on recurring defects. • Coding standards, checklists and peer reviews or inspections. • Training and lessons-learned sessions; process improvement. • Static analysis tools and automated regression tests in CI.	3

Sl. No	Questions	Bloom's Level
	Improvement in quality: Fixing the cause rather than the symptom stops the same defect from recurring each release. Fewer defects mean lower rework cost, more stable releases, better customer satisfaction and a more mature process.	
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability. **Answer:** **Redesigned strategy:** Apply principles such as early testing, risk-based testing, testing is context dependent and defect clustering: • Define a test strategy and plan with scope, levels (unit, integration, system, UAT) and exit criteria. • Use design techniques: equivalence partitioning, BVA and state transition for screens and flows. • Test on a device matrix (OS versions, screen sizes, networks) and include interruption tests (calls, low battery). • Automate regression and track crash logs and metrics. **Reliability:** A standard, repeatable approach covers critical paths and risk areas, finds crash causes earlier and gives consistent results across releases, so the application crashes less and is more dependable.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality. **Answer:** **Testing:** Testing is the process of executing software to find defects and check that it meets requirements. It is done by testers and shows the presence of failures. **Debugging:** Debugging is the process of locating the cause of a defect and fixing it. It is done by developers. **Scenario:** In an online ticket system, a tester finds that the total price is wrong when 3 tickets are booked (testing). The developer traces it to a faulty multiplication in the pricing code and corrects it (debugging). The tester then retests and runs regression tests (testing again). **Roles:** Testing detects and reports; debugging diagnoses and corrects. Clear separation avoids delays, and together they improve quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled. **Answer:** **Cross-browser strategy:** Apply risk-based and equivalence-partition thinking to choose which combinations to test. • Identify target browsers, versions and devices from user analytics (Chrome, Firefox, Safari, Edge, mobile). • Prepare a compatibility matrix and prioritise by market share and risk. • Test layout, JavaScript behaviour, forms, payments and responsive design on each. • Use automation and cloud tools (for example Selenium Grid) and keep a regression suite for each browser.	3

Sl. No	Questions	Bloom's Level
	Handling issues: Log browser-specific defects with browser and version, fix with standards-compliant HTML/CSS and polyfills, and retest. This gives a consistent experience and higher customer satisfaction.	
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability. **Answer:** **Gaps in validation:** The logistics application has inconsistent outputs, which suggests missing validation of inputs such as pincode, weight, address format, date and null values. Testing reveals these gaps by checking valid, invalid, boundary and empty inputs. **Applying testing concepts:** • Equivalence partitioning and BVA for numeric fields such as weight and distance. • Negative testing and data-type checks (letters in numeric fields). • Early reviews of validation rules in requirements. **Improvement in reliability:** With validation tested thoroughly, invalid data is rejected before processing, so routing, billing and tracking outputs stay consistent and the system is dependable.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design. **Answer:** **Testing as an engineering activity:** Engineering needs planning, design and measurement, and testing is no different. Skipping test design leads to repeated, overlapping tests and large gaps. **Improved process:** • Derive test conditions from requirements and write test cases with inputs, steps and expected results. • Apply black-box and white-box techniques and review the test cases. • Track coverage and trace tests to requirements. **Importance of systematic design:** Systematic design gives better coverage with fewer tests, reproducibility, easier maintenance, reuse for regression and objective measurement. As a result, defect detection becomes consistent rather than dependent on luck.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency. **Answer:** **Identifying root causes:** Data inconsistency between modules is typically caused by mismatched data formats, incorrect interface mappings, missing synchronisation or transaction handling, or different interpretations of the same field (for example patient ID). Testers trace data from source module to destination, compare field by field and review interface specifications to find the cause. **Testing principles applied:** Test early at interface level, use integration testing (top-down/bottom-up), consider defect clustering at interfaces, and use data validation and reconciliation checks. **Ensuring consistency:** Integration and end-to-end tests verify that data created in one module (registration) appears correctly in others (billing, pharmacy). Regression tests keep it consistent after each change.	3

Sl. No	Questions	Bloom's Level
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality. **Answer:** **Role of testers across SDLC:** • Requirements: review for testability and ambiguity. • Design: review design and plan integration and system tests. • Coding: prepare test data, support unit testing and static reviews. • Testing: execute tests, report defects and track fixes. • Release and maintenance: acceptance, regression and post-release feedback. **Early involvement improves quality:** Defects are detected at the stage where they are introduced, so there are fewer late surprises and delays. Developers and testers share the same understanding, test planning starts in parallel with development, and the project saves time and cost while delivering better quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures. **Answer:** **Gaps identified:** The failure at peak usage shows missing load, stress and concurrency scenarios. The team tested only normal conditions and did not apply the principle that exhaustive testing is impossible, so risk-based selection should have been used. **Applying principles:** • Use risk analysis to identify critical scenarios such as peak traffic and simultaneous transactions. • Add performance, load, stress and spike tests with realistic user profiles. • Use a requirements traceability matrix to find uncovered requirements. **Better coverage:** Higher coverage of critical and non-functional scenarios exposes bottlenecks (slow queries, memory leaks) before release, so the system continues to work during peak use.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis. **Answer:** **Applying defect repository concepts:** Create a central defect repository or tracking tool (for example Jira or Bugzilla) with a standard defect template. • Record ID, title, module, steps to reproduce, expected and actual result, severity, priority, status, owner and dates. • Define a defect life cycle: New, Assigned, Fixed, Retest, Closed or Reopened. • Attach screenshots and logs. **How documentation helps:** Clear records make defects reproducible, so developers fix them faster. Status tracking gives visibility of progress. Analysis of the data gives metrics such as defect density, leakage and trends, which support root cause analysis and process improvement.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3

Sl. No	Questions	Bloom's Level
	Answer: **Redesigned approach:** Using testing principles (early testing, defects clustering, exhaustive testing is impossible, absence-of-errors fallacy), redesign testing as follows: • Review requirements and plan tests for all transaction types (transfer, deposit, withdrawal). • Design tests with BVA, equivalence partitioning, decision tables and state transitions. • Include negative, rollback, timeout and concurrency tests, plus security tests. • Maintain a regression suite and analyse past failures to focus on high-risk modules. **Reliability:** Structured testing covers critical flows methodically, finds failure causes before release, gives repeatable results and reduces repeated transaction failures.	
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection. **Answer:** **Including edge cases:** Testing only normal inputs ignores the principle that defects cluster at limits. Add edge cases such as minimum and maximum values, empty or null input, zero, negative numbers, very long strings and special characters. **Boundary testing:** BVA tests values just below, on and just above each boundary. For a quantity range 1 to 100, test 0, 1, 2, 99, 100, 101. **Improved detection:** Off-by-one errors (< versus <=) and overflow problems occur at boundaries, so boundary tests find them with a small number of well-chosen test cases and improve defect detection and robustness.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes. **Answer:** **Process-based approach:** Adopt a common test process for all modules, such as the standard test life cycle (planning, design, execution, closure) aligned with models like TMMi or ISO 29119. • Common templates for test plans, test cases and defect reports. • Uniform entry/exit criteria and review checkpoints. • Shared tools and common test standards across teams. **How standardisation helps:** All modules get the same level of rigour, so quality is even. Results are comparable and measurable, new team members learn faster, reuse increases and management can find weak areas quickly.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness. **Answer:** **Applying testing concepts:** The application fails when unexpected inputs are given, so negative and robustness testing was missing. • Use equivalence partitioning with invalid classes (wrong type, empty, too long). • Add error guessing for special characters, SQL or script strings, and spaces. • Test client-side and server-side validation and the error messages shown.	3

Sl. No	Questions	Bloom's Level
	Robustness: With proper testing, the application rejects invalid data with clear messages, does not crash and is protected from misuse such as injection attacks. This makes it dependable and secure.	
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage. **Answer:** **Root cause analysis:** Defect leakage means defects escape to production. Typical root causes: incomplete requirements coverage, weak test design, no regression testing, insufficient environment match, tests skipped under time pressure, and poor communication. **Applying principles:** Use early testing, risk-based prioritisation, traceability to requirements and the principle that testing shows the presence of defects but not their absence. **Preventing leakage:** Add reviews, improve test cases from past leakage, run full regression, use a production-like test environment, and enforce exit criteria before release. Measure leakage (defects found in production / total defects) to track improvement.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency. **Answer:** **Redesigned strategy:** Apply early testing and defect prevention: • Start static testing (reviews, walkthroughs) at requirements and design. • Prepare test cases in parallel with development (V-model). • Use unit testing, continuous integration and risk-based prioritisation. • Track defect detection by phase and set exit criteria. **Efficiency:** Defects found early are cheaper to fix, because the cost of a defect rises sharply in later phases. Early detection reduces rework, shortens schedules, lowers total effort and cost, and improves quality at release.	3